

Opynio Widget Loader — v6.3.0

Carga perezosa SEO-safe + reservación de espacio + timers visibility-aware (con fixes de revisión independiente)

v6.0 →
v6.3.0

Por qué se hizo

El loader v6.0 inicializaba **todos** los widgets en `DOMContentLoaded`, sin importar si estaban dentro o fuera del viewport del host. Cada init disparaba 2 round-trips de red (Supabase + Google Translate) en paralelo al render del cliente. Resultado: degradación del LCP/INP del host y riesgo de penalización por Core Web Vitals.

Cambios aplicados (público: `public/widget.js`)

Componente	v6.0	v6.3.0
Trigger de inicialización	Todos a la vez en <code>DOMContentLoaded</code>	<code>IntersectionObserver</code> con <code>rootMargin: '200px 0px'</code> por widget
Defer post-LCP del host	—	<code>requestIdleCallback(timeout: 1500)</code> antes de cada init
Bypass para crawlers	Sólo render interno	Scheduler respeta <code>IS_BOT</code> → render inmediato sin observer
Reservación de espacio (CLS)	—	<code>min-height</code> por tipo aplicado antes de hidratar
Timers de carruseles	<code>setInterval</code> eternos	<code>visibilityAwareInterval</code> : pausa al salir de viewport y con <code>document.hidden</code> ; teardown completo en <code>stop()</code> (IO + listener + interval)
MutationObserver	<code>document.body subtree</code> , dispara siempre	Filtra <code>.opynio-widget</code> , debounce 50 ms; limpia recursos en <code>removedNodes</code> (SPAs)

Fixes post-revisión (v6.1 → v6.3.0)

v6.1.1 — leaks detectados por revisión independiente

- **Leak de listeners `visibilitychange`**: cada re-arm del ticker dejaba un listener pegado a `document`. `stop()` ahora hace `removeEventListener`.
- **Leak de `IntersectionObserver` internos**: cada re-arm creaba un IO nuevo sin desconectar el anterior. `stop()` ahora hace `io.disconnect()`.
- **Cleanup en SPAs**: el `MutationObserver` ahora itera `removedNodes` y libera `lazyObserver` + ticker asociado.
- **Race resuelto** entre IO y `visibilitychange`: `tick()` filtra, el listener llama `start()` incondicional.
- **Test page**: `document.hidden` restaurado con `delete`; checks rAF-throttled durante scroll.

v6.1.2 — endurecimiento adicional

- **BOT_REGEX ampliado**: añadidos `Mediapartners-Google` (AdSense), `Chrome-Lighthouse` (PageSpeed Insights), `TelegramBot` y `Pinterest`. Lighthouse en cliente ya no califica al host con un estado lazy a medio cargar.
- **reserveSpace respeta CSS externo**: lee `getComputedStyle(el).minHeight` antes de aplicar el default, así un host con `.opynio-widget { min-height: 200px }` en su CSS gana.
- **Chrome prerender**: `document.prerendering === true` fuerza render eager (sin observer) — el page activado al usuario ya está listo.

v6.2.0 — limpieza SEO definitiva

- **JSON-LD `AggregateRating` eliminado del host**: Google ignora el rich result self-serving para `LocalBusiness/Organization` desde 2019. La structured data canónica vive sólo en `web.opynio.com`.
- **Bot-path centralizado para los 9 widgets**: cuando `IS_BOT`, `initWidget` llama a `renderBotSafe()`. Bots ven sólo brand + estrellas + count + anchor canónico — nunca el contenido completo de las reseñas. Elimina duplicate content entre hosts.
- **`rel="noopener nofollow"` en los 10 anchors `target="_blank"`** y en el `window.open` del sidebar. Cierra tabnabbing y neutraliza el patrón de link scheme.

v6.3.0 — UX floating + cache de traducciones

- **Floating con botón cerrar:** × discreto en la esquina del pill. Cliente lo cierra → guarda dismissal en `localStorage` con TTL de 7 días, y `scheduleWidget` lo respeta sin siquiera hacer fetch.
- **data-position:** `bottom-left` (default), `bottom-right`, `top-left`, `top-right`. El host elige la esquina para no chocar con su cookie banner / chat / scroll-to-top.
- **Cache persistente de traducciones:** `translateReviews` ya no dispara 2×N requests en cada page load. Cache en `localStorage` con TTL de 7 días, fallback in-memory si la cuota se llena. Elimina llamadas redundantes al endpoint `gtx` no oficial.
- **UI string `close`:** añadido a los 13 idiomas hardcoded (es, en, fr, de, it, pt, ca, zh, ja, ko, nl, ru, ar) — sin caer al fallback español en hosts internacionales.

Veredicto SEO post v6.3.0: el widget cumple la regla maestra "*neutral o positivo, nunca negativo*" en CWV, indexación, structured data y autoridad de enlace. Acción del usuario (cerrar floating) deja de ser señal negativa de UX porque ahora la persistencia respeta su elección.

Shadow DOM — diferido conscientemente

La migración a Shadow DOM (aislamiento total del CSS, eliminación de los `!important`) **no se incluye en este PR**. Razón: requiere refactor de cada renderer (cambiar `el.innerHTML` / `el.querySelector` por raíz nueva, convertir `:root` a `:host` en CSS, ruta paralela del bot path) sin upside SEO bloqueante. Hacerlo deprisa introduce regresiones en hosts ya en producción. Plan concreto en `docs/12-WIDGET-SHADOW-DOM-PLAN.md` (próximo PR).

Decisiones clave

rootMargin 200 px Pre-renderiza el widget justo antes de entrar al viewport. El usuario ve contenido, no spinner. Recomendado por Google Search Central porque no depende de scroll.	Bypass de Googlebot Google Search Central: "<i>Googlebot doesn't scroll, scroll events never fire</i>". <code>IS_BOT</code> salta el observer y renderiza directo, garantizando indexación.
requestIdleCallback timeout 1500 ms El widget no compite con el LCP del host. Si el navegador no está idle a tiempo, el timeout fuerza el render. Fallback a <code>setTimeout</code>.	min-height por tipo Cada widget reserva su espacio antes de pintar contenido (badge 90px → wall 560px). CLS aportado ≈ 0. Floating omitido (es position:fixed).

Método de validación

Página de test sin dependencias: `public/widget-test.html` — se sirve con `npm run dev` en `http://localhost:3000/widget-test.html`.

- Mockea `fetch` a `widget-proxy` y a Google Translate (corre offline).
- Monta 5 widgets a distintas alturas + 1 floating.
- Panel sticky con 10 invariantes evaluadas en vivo: lazy schedule, espacio reservado, eager-init guard, errores JS, CLS aportado, pausa por visibility, bypass de bot.
- Modo `?bot=1` → fuerza `navigator.userAgent` a Googlebot y valida bypass.
- Botón "Ocultar pestaña" → simula `document.hidden` 5 s y compara `transform` de carruseles.

Validación oficial pendiente (manual, post-deploy en staging): Google Search Console → URL Inspection → *Test Live URL* · Rich Results Test · PageSpeed Insights antes/después.

Pendientes para próximos PRs

Los tres bloqueadores SEO (JSON-LD self-serving, rel en anchors, bot-path en los 9 widgets) quedaron cerrados en v6.2.0. UX del floating y cache de traducciones cerrados en v6.3.0. Sólo queda Shadow DOM, sin impacto SEO bloqueante:

Sev.	Hallazgo	Por qué
	Shadow DOM para los renderers humanos	Aislar CSS del host, eliminar los <code>!important</code> heredados, blindar contra temas de WordPress. Refactor real (renderers + CSS <code>:root</code> → <code>:host</code>) — su propio PR con su propio testeo.

Archivos modificados / creados

- **Modificado:** `public/widget.js` — scheduler lazy + carruseles visibility-aware + bot-path centralizado + JSON-LD eliminado del host + rel en anchors + UX floating + cache traducciones (v6.0 → v6.3.0)
- **Nuevo:** `public/widget-test.html` — página de test diagnóstica con mock de fetch y panel de invariantes
- **Nuevo:** `docs/11-WIDGET-LAZY-LOADING-V6.1.md` — documentación técnica completa del cambio
- **Nuevo:** `docs/12-WIDGET-SHADOW-DOM-PLAN.md` — plan concreto del refactor de Shadow DOM (próximo PR)